

Data Engineer — Easy Guide

Crash course before the presentation · Retail Demand Forecasting (Walmart M5)

In one sentence

I take three messy Walmart files and turn them into one clean table with 59 million rows, then prove nothing got broken.

The 60-second version

- The data is the public **Walmart M5** dataset: **30,490** product-store combinations, **1,941 days** of history (2011-01-29 → 2016-05-22).
- It arrives as **three separate tables**: sales, calendar, prices.
- I reshape and join them into **one long table**: **59,181,090 rows**.
- One row = **one product, in one store, on one day**.
- **68% of all cells are zero** — most products don't sell every day.
- I audited everything and **found no defects**, so I fixed nothing.

The picture

```
sales.csv      calendar.csv      sell_prices.csv
|              |              |
+----- AUDIT everything -----+
|
v
MELT   wide (1,941 day-columns) -> long (1 row per day)
|
v
JOIN   + date, weekday, events, SNAP
       + price for that store/item/week
|
v
sales_long_full.parquet      59,181,090 rows      287 MB
|
v
feature engineering -> model
```

Words you must be able to explain

Word	Say this
CSV	Comma-Separated Values — a plain text table, one line per row
Parquet	A compressed file format that stores data by column; much smaller and faster than CSV, and it remembers data types
DuckDB	A database that runs inside our app instead of on a separate server
Row / column	A row is one record; a column is one named field
Wide vs long	Wide = one column per day. Long = one row per day. Models need long
Join	Gluing columns from another table on by matching a key
Null vs zero	Null = we don't know. Zero = we know it was none. Very different
SNAP	Supplemental Nutrition Assistance Program — a US food benefit. Our calendar flags which days it could be spent
Intermittent demand	Demand that is zero on most days — that's our dataset

Word	Say this
MD5 / SHA-256	A fingerprint of a file. If one byte changes, the fingerprint changes

Numbers to memorise

Series (store-item)	30,490 (3,049 items x 10 stores)
Days of history	1,941
Rows after reshaping	59,181,090
Total units sold	66,927,173
Zero cells	40,241,819 = 68.00%
Stores / states	10 / 3 (CA, TX, WI)
Rows with no price	12,299,413 = 20.78% (expected, not a defect)
Processed file	sales_long_full.parquet, 287 MB

The two findings that matter

- 1. 68% zeros.** Most products don't sell every day in every store. We kept every zero — a zero is a real observation. Deleting them would teach the model that everything sells constantly.
- 2. 20.78% of rows have no price — and that is correct.** A product has no price record before it was introduced to that store. So the gap is *information*, not damage. We leave it empty rather than filling in a number that never existed.

If a judge says "20% missing prices sounds bad" — that is your moment. It is not missing data. It marks products that weren't stocked yet.

What we checked (and found nothing wrong)

Missing values · duplicate IDs · negative sales · calendar gaps · event name/type consistency · SNAP values · price validity · duplicate price rows · do the tables agree with each other.

Every check returned 0 problems. So we fixed nothing — deliberately. We documented that instead of inventing repairs.

How we prove nothing changed

- **MD5 checksums** on all five raw files.
- **46 integrity and leakage checks** in `01_foundation_check.py`.
- A **SHA-256 manifest over 522 protected files**; last run: 0 deleted, 0 modified.

Files to name if asked

- `03_RESEARCH/data/raw/` — the five original CSVs, never written to
- `03_RESEARCH/data/processed/sales_long_full.parquet` — the joined table
- `03_RESEARCH/data/processed/PROCESSING_REPORT.md` — what we checked and found
- `03_RESEARCH/pipeline/config.py` — all paths and dataset constants
- `01_PROJECT/backend/scripts/build_product_db.py` — builds the data the app serves

How to explain it to a judge

"My part is the data. We used Walmart's M5 dataset — 30,490 product-store combinations across 1,941 days. It comes as three separate files: sales with one column per day, a calendar, and weekly prices. I reshaped the sales into long format and joined the calendar and prices onto it, which gives about 59 million rows where one row is one product, in one store, on one day.

I audited the raw files first — missing values, duplicates, negatives, calendar gaps, price validity, whether the tables agree. Everything came back clean, so we deliberately fixed nothing.

Two things stand out: 68% of cells are zero, because most products don't sell every day — that's what makes this an intermittent demand problem. And 20% of rows have no price, which is expected: those products weren't stocked in that store yet."

Likely questions, short answers

What does one row mean? One product, one store, one day — with sales, calendar and price attached.

What cleaning did you do? None was needed. Every audit check returned zero problems, and we documented that rather than inventing fixes.

Why not drop the zeros? A zero is a real measurement. Dropping them would destroy the exact behaviour we're forecasting.

Why not fill in the missing prices? Because they're not missing — the product didn't exist in that store yet. Filling them would invent history.

How do you know the data is intact? MD5 checksums, a 46-check integrity suite, and a SHA-256 manifest over 522 protected files.

Why Parquet instead of CSV? Smaller, faster, and it preserves data types. For the app we go further: a DuckDB layer of 31.7 MB instead of 287 MB, answering a single-series query in 8–13 ms instead of ~320 ms.

Wide vs long — the reshape, concretely

This is the single transformation people ask about, so be ready to draw it.

Before (wide) — one row per product-store, 1,941 columns of days:

id	d_1	d_2	d_3	...	d_1941
FOODS_3_090_CA_3	0	2	1	...	4

After (long) — one row per day:

id	day_idx	sales	date	wday	snap	sell_price
FOODS_3_090_CA_3	0	0	2011-01-29	6	0	(none)
FOODS_3_090_CA_3	1	2	2011-01-30	7	0	(none)
FOODS_3_090_CA_3	2	1	2011-01-31	1	1	1.58

One wide row of 1,941 day-columns becomes 1,941 long rows. Do that for all 30,490 series and you get 59,181,090 rows.

Notice the last column: early rows have **no price**, because that product had not been introduced to that store yet. That is the 20.78%.

The second dataset — what the app actually uses

The 287 MB research file is for modelling. The running application does **not** read it. We build a smaller product layer from the frozen artefacts:

```
01_PROJECT/backend/scripts/build_product_db.py
|
v
01_PROJECT/backend/data/
  product.duckdb      small tables (series, forecast, calendar, accuracy)
  history.parquet     59.2M rows of actuals, sorted by series    31.7 MB
  backtest.parquet    6.8M rows of backtest results              78.4 MB
```

Why bother? Two measured reasons:

	Research parquet	Product sidecar
Size of the history	287 MB	31.7 MB
Time to read one product's history	~320 ms	8–13 ms

The research file repeats the item and store name on all 59 million rows; the product file keeps only what the app needs and sorts by series, so a lookup touches a tiny slice of the file.

What I deliberately did NOT do

- I did **not** delete zero rows — they are real observations.
- I did **not** fill in missing prices — the gap is meaningful.
- I did **not** modify the raw files — they are opened read-only, always.
- I did **not** "fix" anything, because the audit found nothing broken.

Being able to say **why you did nothing** is stronger than listing fake cleaning steps.

30-second revision

30,490 series · 1,941 days · 59,181,090 rows · 68% zeros · 20.78% no price (expected) · three tables joined into one · audited clean, fixed nothing · verified with checksums and 522-file hash manifest.